

Java Programming Exercises

HashMaps and Tree Structures

Exercise 1 — Student Management

Build a Java program that manages student records in memory, backed by a HashMap keyed by student ID.

Student class

Fields: `studentId` (String), `fullName` (String), `className` (String), `averageScore` (double).

Storage

HashMap<String, Student>, keyed by `studentId`.

Functions

- Add a new student: check `containsKey()` first and reject (or warn about) a duplicate ID before inserting — `HashMap.put()` silently overwrites whatever was already stored under that key, so adding a student with an ID that's already in use would quietly erase the existing record rather than raising any kind of error.
- Find a student by ID: `get()` the key, display the student if present, otherwise print a not-found message.
- Update a student's average score: look the student up by ID and set the new score via the setter on the existing object — don't replace the map entry with a brand-new Student that has blank name and class fields.
- Remove a student by ID: `remove()`, and report whether anything was actually removed.
- Print all students with `averageScore >= 8.0`: iterate `map.values()` and filter.

Exercise 2 — Word Frequency Counter

Build a Java program that reads a block of text from the keyboard and reports its most frequent words.

Input

Read lines from the keyboard until the user enters a blank line — that's the signal that the text block is finished.

Counting

- Lowercase everything before counting, so "The" and "the" count as the same word.
- Split on non-word characters rather than on whitespace alone, so punctuation attached to a word ("word," or "word.") doesn't create a separate token from the plain word.

- Use `HashMap<String, Integer>`: for each token, increment its count, or insert it with count 1 if it's new.

Output

- Print the 5 most frequent words, in descending order of count. If there's a tie for the 5th spot, showing any one of the tied words to round out the top 5 is fine — no specific tie-breaking order is required.
- If fewer than 5 distinct words were entered, print as many as exist.

Exercise 3 — Warehouse Inventory Management

Build a Java console application that manages a warehouse's product inventory, backed by a `HashMap` keyed by product ID.

Product class

Fields: `productId` (`String`), `productName` (`String`), `stockQuantity` (`int`), `sellingPrice` (`double`).

Storage

`HashMap<String, Product>`, keyed by `productId`.

Menu

- 1 — Receive stock
- 2 — Ship stock out
- 3 — Search for a product by name
- 4 — Print low-stock products
- 5 — Calculate total inventory value

Option 1 — Receive stock

Prompt for a product ID and a quantity received; reject a quantity that isn't greater than 0. If the ID already exists, increase its stock by that quantity. If it doesn't exist yet, register it as a new product: also prompt for a name and selling price, and store it with the received quantity as its starting stock.

Option 2 — Ship stock out

Prompt for a product ID and a quantity to ship; reject a quantity that isn't greater than 0. If the ID doesn't exist, or the requested quantity exceeds current stock, report an error and don't change anything. Otherwise subtract the quantity from stock.

Option 3 — Search by name

Case-insensitive substring match against every product's name; print every match, or a "no matches" message if there are none.

Option 4 — Low-stock list

Print every product with `stockQuantity < 10`.

Option 5 — Total inventory value

Sum `stockQuantity × sellingPrice` across every product.

Exercise 4 — Shopping Cart Management

Build a Java program that manages the contents of a shopping cart, backed by a `HashMap` keyed by product ID.

CartItem class

Fields: `productId (String)`, `productName (String)`, `sellingPrice (double)`, `quantity (int)`.

Storage

`HashMap<String, CartItem>`, keyed by product ID.

Functions

- Add a product to the cart, with a quantity; reject a quantity that isn't greater than 0. If the product's ID is already a key in the cart, increase the existing `CartItem`'s quantity by the new amount rather than creating a second entry; otherwise insert a new `CartItem`.
- Decrease the quantity of a product by a given amount: if the product isn't currently in the cart, report that rather than crashing. Otherwise, if the resulting quantity would be 0 or less, remove the item from the cart entirely rather than leaving a zero-quantity entry behind — this one operation handles both shrinking the quantity and fully removing the item.
- Calculate the total cost: sum `sellingPrice × quantity` across every cart entry.
- Print the cart's contents sorted by product name, alphabetically.

Exercise 5 — Employee Salary Directory

Build a Java program that manages a directory of employee salaries, backed by a `HashMap` keyed by employee name.

Storage

- `HashMap<String, Double>`, keyed by employee name. Salary is declared as `double` rather than `Integer`, since that doesn't force every salary to be a whole number.
- Because name is the key, two employees who happen to share the exact same name can't both be stored — adding the second one will silently overwrite the first's salary via the normal `Map.put()` behavior. That's an accepted limitation of keying by name rather than by a unique employee ID, not something to work around unless you want to change the key type.

Functions

- Add employee data (name and salary) for at least 6 employees.

- Search for an employee's salary by name; print "Employee not found" if the name isn't a key in the map.
- Update the salary for a given name (optional).
- Remove an employee from the map by name.
- Print every employee and their salary.

Exercise 6 — Contact List

Build a simple contact list that stores phone numbers by name, backed by a HashMap.

Storage

HashMap<String, String>, name (key) → phone number (value).

Functions

- Add, remove, and search for contacts by name.
- Duplicate phone numbers under different names are already supported without any extra code — a Map's values are never required to be unique, only its keys are, so two different name-keys can already hold the identical phone-number value with a plain HashMap.
- Optional: sort the entries by name and display the sorted list (e.g. copy the key set into a List, sort it, then look up and print each name/number pair in that order).

Basic usage

```
HashMap<String, String> contacts = new HashMap<>();
contacts.put("Alice", "123-4567");
contacts.put("Bob", "987-6543");

// Searching by key (name):
System.out.println(contacts.get("Alice")); // Output: 123-4567

// Removing a contact:
contacts.remove("Bob");
```

Exercise 7 — Character Frequency Counter

Build a Java program that reads a string and reports how many times each character appears in it.

Requirements

- Read a string, and use HashMap<Character, Integer> to count how many times each character appears.

- This one is case-sensitive and counts every character as-is, including spaces and punctuation — 'A' and 'a' are different keys here, and nothing gets stripped out before counting.
- Print the resulting map; any order is fine, since none is specified.

Exercise 8 — Most Frequent Element in an Array

Build a Java program that reads an array of integers and reports its most frequently occurring value.

Requirements

- Read an array of integers. If the array is empty, there is no most-frequent element; print an appropriate message instead of computing one.
- Use `HashMap<Integer, Integer>` to count how many times each value appears.
- Find the entry with the highest count; if more than one value ties for the highest count, reporting any one of them is fine.
- Print the most frequent value and how many times it occurs.

Exercise 9 — Simple LRU Cache (Bonus / Advanced)

Build a fixed-capacity cache that evicts its least-recently-used entry whenever it needs room for a new one.

Requirements

- Implement a fixed-capacity cache using Least-Recently-Used eviction: when a full cache needs to store a new key, it discards whichever currently-stored key was accessed least recently to make room. Reject a capacity that isn't greater than 0.
- Combine a `HashMap` (for $O(1)$ key lookup) with a doubly linked list (to track access order cheaply — moving a node to the "most recently used" end, or removing the "least recently used" end, are both $O(1)$ on a doubly linked list, unlike an `ArrayList`, where removing from an arbitrary position is $O(n)$).

Input

- The cache capacity, then a sequence of access requests. Represent each request as an operation plus a key (and, for a write, a value) — e.g. `get(3)` or `put(3, "value")`.
- A `get` or `put` on a key that's already present both count as "using" that key: move it to the most-recently-used end. A `put` on a brand-new key when the cache is already at capacity evicts the least-recently-used entry first.

Output

After each access, print the cache's current contents ordered from most-recently-used to least-recently-used, so the eviction behavior is visible as it happens.

Exercise 10 — Student Grade Lookup (HashTable)

Build a Java program that looks up a student's letter grade by name.

Requirements

- Store at least 5 students as name → grade pairs. HashMap is the right default choice here; the older Hashtable class is synchronized (thread-safe), but that costs performance for no benefit in a single-threaded console program like this one — use it only if the exercise specifically asks you to.
- Represent grade as a String (e.g. "A", "B+") — the everyday meaning of "grade" — though a numeric score is an equally reasonable alternative if you'd rather.
- Prompt for a name from the keyboard and look it up; print "Not found" if it isn't a key in the map.

Exercise 11 — Build a Tree from Categories and Subcategories

Build a Java program that converts a flat list of category/subcategory pairs into a tree structure and prints it.

Convert a list of [category, subcategory] pairs into a tree, given for example:

```
String[][] categories = {
    {"Electronics", "Phones"},
    {"Electronics", "Laptops"},
    {"Home", "Furniture"},
    {"Home", "Appliances"}
};
```

TreeNode structure

Fields: String data and List<TreeNode> children — a List rather than a fixed-size array, since children get added one at a time as new subcategories are discovered.

Building the tree

- Keep a lookup (e.g. a Map) from category name to its TreeNode, separate from the top-level structure itself. For each [category, subcategory] pair: if a TreeNode for that category doesn't exist yet in the lookup, create one and add it to the top-level list; then add a new child TreeNode for the subcategory under it.
- The important part to get right: "Electronics" appears in two separate rows of the input. Both rows have to add their subcategory under the same Electronics node — checking the lookup before creating a new category node is what prevents two separate, duplicate "Electronics" nodes from ending up side by side in the output.
- There's no single shared root object holding "Electronics" and "Home" as children — keep a list of top-level category nodes instead, and print by looping over that list. A shared invisible root would need its own label suppressed specially just to make the output look right, which the list-of-top-level-nodes

approach avoids entirely.

Printing

Use the following recursive print function:

```
void printTree(TreeNode node, String indent) {
    System.out.println(indent + node.data);
    for (TreeNode child : node.children) {
        printTree(child, indent + "- ");
    }
}

// Call this once per top-level category:
for (TreeNode topLevel : topLevelCategories) {
    printTree(topLevel, "");
}
```

Expected output

For the example categories above:

```
Electronics
- Phones
- Laptops
Home
- Furniture
- Appliances
```

Exercise 12 — Binary Search Tree (BST) with Removal

Build a binary search tree from a fixed sequence of integers, and implement node removal.

Build the tree

Insert [45, 23, 67, 12, 34, 89, 50] into a BST one at a time, in that order, using standard BST insertion: a value less than the current node goes left; a value greater than or equal to the current node goes right.

Removal — the three cases

BST removal has three distinct cases depending on the node being removed:

- The node is a leaf (no children): just detach it from its parent.
- The node has exactly one child: splice it out by connecting its parent directly to its one child.
- The node has two children: find its in-order successor (the smallest value in its right subtree — keep following left children from the right child until there isn't a left child left), copy that value into the node being "removed," then remove the successor node from its original spot instead. That removal is guaranteed to fall into one of the two simpler cases above, since the smallest value in a subtree can

never have a left child.

Expected output

- Pre-order traversal (root, left, right) of the tree built from the insertion order above: 45, 23, 12, 34, 67, 50, 89.
- 34 turns out to be a leaf in this particular tree, so removing it is the simplest case — just detach it from its parent (23). Pre-order after removal: 45, 23, 12, 67, 50, 89.

Exercise 13 — Balanced Binary Search Tree

Build a balanced binary search tree directly from a sorted array.

Building from a sorted array

- From [10, 20, 30, 40, 50, 60, 70, 80]: recursively pick the middle element of the current range as the root, then recurse on the left and right halves for the left and right subtrees.
- With an even-length range there are two middle elements to choose between; pick $\text{mid} = (\text{lo} + \text{hi}) / 2$ (integer division) consistently — that is, whichever element that formula lands on, every time, rather than alternating conventions partway through. The other common convention, $\text{mid} = (\text{lo} + \text{hi} + 1) / 2$, is equally valid and produces an equally balanced tree, just a structurally different one — the expected output below assumes the first formula.

Expected output

- Using $\text{mid} = (\text{lo} + \text{hi}) / 2$. In-order: 10, 20, 30, 40, 50, 60, 70, 80 — always sorted for any valid BST shape, so this confirms the BST property holds but doesn't by itself confirm the tree is balanced the way you intended.
- Optional: also print the pre-order and post-order traversals. Pre-order: 40, 20, 10, 30, 60, 50, 70, 80. Post-order: 10, 30, 20, 50, 80, 70, 60, 40.

Exercise 14 — Balanced Binary Tree from Sequential Insertion

Build a binary search tree through ordinary one-at-a-time insertion, using an input order that happens to produce a naturally balanced tree without any special construction logic.

Building the tree

Insert [50, 30, 70, 20, 40, 60, 80] one at a time, in that order, using standard BST insertion: a value less than the current node goes left; a value greater than or equal to the current node goes right.

Expected output

In-order traversal: 20, 30, 40, 50, 60, 70, 80.